

13 — Failure Modes

Objetivo: documentar cómo falla Puglit. No *si* falla — *cómo*. Todo sistema que orquesta modelos chicos y locales para escribir código de cero rompe de maneras concretas y repetibles. Este capítulo cataloga los modos de falla REALES (observados en el litmus test y cableados como defensas en el código), explica por qué ocurren, y da los *recovery playbooks* — la secuencia exacta para volver a un estado verde. Cada entrada cita el archivo y la función que la detecta o la repara, así no es un manual de buenos deseos sino un mapa contra el código que corre hoy.

La tesis de diseño es simple: **un compile verde no es software que funciona**. Modelos chicos (`qwen2.5-coder` , `deepseek-coder-v2` , `devstral` sobre Ollama) producen código que pasa `tsc` y crashea en runtime, SQL que parsea y referencia tablas que nadie creó, y JSX que es válido como string JSON pero inválido como TSX. Puglit asume eso y construye gates en cascada. Las secciones que siguen recorren cada capa donde el sistema puede caer, de la más concreta (bugs reales del generado) a la más sistémica (scaling, seguridad, dependencias).

13.1 — Known Failure Modes (los reales del litmus test)

El litmus test **Stayforge** (clon de Airbnb: 80 archivos, 8 tablas, `EXCLUDE USING gist` anti-doble-booking, pricing en centavos enteros, refund por snapshot de política, reviews doble-ciego) expuso **7 bugs reales invisibles a un compile verde**. No son hipótesis: son la razón de que exista el *runtime gate* en `web/scripts/build-local.mjs` . Cada uno tiene hoy una defensa determinística cableada.

Tabla — Known Failure Modes

#	Failure mode	Síntoma	Por qué pasa	Defensa cableada	Dónde
1	<code>pool()</code> vs <code>pool</code> object	<code>TypeError: pool is not a function</code> → 500 en runtime, <code>tsc</code> no lo ve	El modelo "recuerda" un cliente DB invocable; el spine exporta <code>export const pool = new Pool()</code> (un objeto, no una factory)	<code>ensureSpineImports()</code> prepone <code>import { pool } from "@lib/db"</code> ; el smoke test atrapa el 500 si la firma de uso es errónea	<code>build-local.mjs:133-149, :387-411</code>
2	Módulo no cableado	Import de <code>@lib/<x></code> que no existe → <code>ReferenceError</code> / módulo faltante	El modelo escribe lógica asumiendo un helper que nunca generó ni es del spine	<code>consistencyScan()</code> flag <code>missing-import</code> (med) contra el set de archivos generados + libs del spine	<code>swarm-checks.ts:48-53</code>
3	Hardcoded table name	El módulo (p.ej. rentals) quereaa <code>bookings</code> cuando la tabla detectada se llama <code>reservations</code>	Verticales escritos con un nombre de tabla fijo en vez de parametrizado	Módulos PARAMETRIZADOS al nombre de tabla detectado (zero hardcoding); <code>consistencyScan</code> flag <code>phantom-table</code> si la referencia no existe	<code>swarm-checks.ts:31-46, module-registry.ts</code>
4	JSON over-escaped JSX	<code>Expected unicode escape</code> , cada página 500	El LLM devuelve fuente JSON-escapada (<code>className="\\"... \\"</code>) que es válida como string pero inválida como TSX	<code>unescapeJsx()</code> desescapa cuando detecta la firma <code>=\\" / \\\n / className=\\</code>	<code>build-local.mjs:152-157</code>
5	Spine sin <code>node_modules</code>	Turbopack: <code>Cannot find module 'next'</code> ; toda app servida con dangling deps	El spine se ensambla sin sus deps instaladas	Chequeo <code>next + vitest</code> presentes → <code>npm install</code> en el spine una vez; <code>node_modules</code> se replica con <code>cp -al</code> (hard-links, mismo FS)	<code>build-local.mjs:169-175</code>
6	Turbopack root	Next infiere el root EQUIVOCADO (lockfile del padre) → no bootea → smoke test falla por razón ajena a la app	<code>DIR</code> está anidado dentro del repo de Puglit	<code>next.config.ts</code> pinea <code>turbopack.root + outputFileTracingRoot</code> a <code>DIR</code> ; <code>node_modules</code> es un dir REAL adentro (no symlink cross-FS, que Turbopack rechaza)	<code>build-local.mjs:38-44, :185-188</code>

#	Failure mode	Síntoma	Por qué pasa	Defensa cableada	Dónde
7	pgvector ausente	<code>type "vector" does not exist</code> al cargar SQL; o embeddings que no persisten	El RAG/diary usa <code>pgvector</code> y el pod/DB no tiene la extensión	Embeddings se guardan como JSONB cuando <code>vector</code> no está; el diary tolera <code>embedding IS NULL</code> y cae a recencia	<code>progression.ts:159-175</code> , <code>brain-sync.ts:23-25</code>

13.1.1 — La cadena de gates que los atrapa

Ninguno de estos 7 lo atrapa `tsc`. Por eso el pipeline local corre, en orden:

1. **SQL load contra DB fresca** (`loadAppSchemaRepairing`, `build-local.mjs:216-241`) — carga `001_core` → `002_auth` → `003_records` → `app.sql` → `seed.sql` contra una DB recién creada cada intento, con `ON_ERROR_STOP=1`. Repara DDL hasta 5 veces (orden de dependencias, sin ENUM inline, sin tablas inexistentes). Si no converge, sigue: "la app puede correr con tablas parciales".
2. **`tsc --noEmit` con anti-oscilación** (`repairTs`, `:261-292`) — alimenta cada archivo con error de vuelta al modelo; guarda el **best-seen snapshot** (menos errores) y para si 2 rondas no mejoran, restaurando el mejor. Evita el loop infinito de "arregla A, rompe B, arregla B, rompe A".
3. **Runtime gate / smoke test** (`smokeTest`, `:387-411`) — **el chequeo que importa**. Bootea `next dev`, camina el árbol `app/`, hace `GET` a cada página estática (side-effect-free, salta `[dynamic]`, `(groups)` y `api`) y asserts **0 respuestas 5xx**. Ahí es donde los 7 bugs de Stayforge se hicieron visibles.

Lección de diseño: los bugs #1, #4, #6 sólo aparecen al bootear. Un sistema que entrega "el código compila" entrega software roto. El runtime gate es la diferencia entre una demo y una app.

13.2 — Agent Failures

Los agentes son modelos chicos locales. Fallan de tres formas, todas manejadas sin tirar el pipeline.

Tabla — Agent Failures

Failure	Síntoma	Manejo	Dónde
Malformed JSON	El modelo devuelve JSON inválido / fuera de schema	<code>chatJSON()</code> valida contra JSON-Schema; el call-site envuelve en <code>try/catch</code> y devuelve <code>null</code> → el equipo se saltea esa pieza esa ronda	<code>tournament.ts:90-107</code> , <code>adversarial-review.ts:70</code>
Refusal / output vacío	El modelo se niega o devuelve <code>< N</code> chars	Toda reparación chequea longitud mínima antes de aceptar (<code>out.length > 60</code> , <code>> 40</code> , <code>> 30</code>); por debajo, NO sobrescribe — conserva el archivo previo	<code>swarm-repair.ts:30</code> , <code>build-local.mjs:283</code> , <code>:340-341</code>
Model timeout / no pulleado	El modelo del team no está bajado o cuelga	Graceful council degradation: el team reintenta con <code>MODELS.code</code> y <i>sigue compitiendo</i> (marca <code>fallback de <modelo></code>); si también falla, ese team se omite pero el torneo continúa	<code>tournament.ts:62-71</code>

13.2.1 — Por qué nunca un agente solo bloquea el build

El patrón es invariante: **un fallo de agente degrada, no aborta**. En `build-local.mjs` la función `ask()` "NEVER throws — a failed call just skips that fix this round". Usa streaming HTTP para ser inmune al `headersTimeout` de 300s de undici incluso cuando una reparación grande tarda minutos en una caja de 8GB. El único punto donde el sistema sí aborta es cuando **cero** designs salieron del torneo (`runDivergence` → `{ ok: false, error: "no designs (¿modelos del council bajados?)" }`, `tournament.ts:170`) — y eso es un problema de infraestructura (modelos no pulleados), no de un agente.

13.3 — Judge Failures

El jurado (el "Gran Jurado" multi-modelo de `judgeBlueprints`) es la señal de fitness del torneo genético. Si el jurado se cae, la selección se cae — salvo que haya un circuit breaker. Lo hay.

13.3.1 — La cadena jury-down → circuit breaker → draft

```
judgeBlueprints() → por cada juror: judgeOnce().catch(() => null)
├── ≥1 veredicto válido → promedia por área, blend 60% juez / 40% objetivo
│                       winner = voto mayoritario (desempate: overall ag
└── 0 veredictos válidos → CIRCUIT BREAKER
                        recordMetric("judge_failed", 1)
                        winner = blueprint MÁS COMPLETO (routes+tables)
                        scores → draft mode 60/60/60/60
                        critique: "draft mode (jurado no disponible)"
```

`tournament.ts:121–133`. La clave del breaker: **no bloquea el pipeline**. Si el jurado entero falló (todos los jurors devolvieron `null`), el sistema degrada a *draft mode* y elige al equipo con el blueprint más completo por conteo de rutas + tablas. El build sigue.

Tabla — Judge Failures

Failure	Detección	Recovery
Un juror falla	<code>judgeOnce().catch(() => null)</code> por juror	Se promedia sobre los jurors que sí votaron
Jurado entero cae	<code>if (!verdicts.length)</code>	Circuit breaker → draft mode, winner por completitud, métrica <code>judge_failed</code>
Juez ruidoso (acuerda poco)	<code>judge_agreement</code> = fracción de jurors que votaron al ganador mayoritario	Se registra para el scorecard; el blend 60/40 con <code>objectiveScore</code> ya amortigua el ruido
Juez cariñoso (no discrimina)	Rúbrica explícita con anclas (90-100 ships / <50 broken) + postura adversarial en el system prompt	Penaliza tablas alucinadas, rutas dead-end, falta del core action

13.3.2 — Por qué el blend 60/40 importa

El juez LLM es ruidoso por construcción. El `overall` de cada equipo no es el veredicto crudo del juez sino `Math.round(0.6 * judgeOverall + 0.4 * obj)` (`tournament.ts:153`), donde `obj` es un `objectiveScore` determinístico y medible del blueprint. Aunque el jurado se equivoque, el 40% objetivo ancla la selección a algo verificable. El `judge_agreement` no cambia la decisión pero queda como señal de "¿cuánto le creemos a este fitness?" para el análisis post-hoc.

13.4 — Evolution Failures

La evolución es donde Puglit acumula conocimiento (diary de lecciones, skills entrenables). Es también donde un sistema que aprende puede aprender lo **incorrecto**. Dos modos de falla y dos gates.

13.4.1 — Lesson poisoning → quality filter (anti-poisoning)

El riesgo: una ronda mala escribe una lección mala al diary; esa lección se inyecta como few-shot en el próximo build y propaga el error. O una lección de otro dominio (fintech → health) se aplica por error.

El gate (`relevantLessons`, `progression.ts:154-175`):

- **Quality floor:** `COALESCE(d.quality, 60) >= 45` — una ronda mala/envenenada tiene quality baja y queda fuera del recall.
- **Outcome filter:** `COALESCE(d.outcome, 'unknown') <> 'failure'` — una lección de una ronda que falló el gate NUNCA se recupera.
- **Relevance floor:** `RELEVANCE_FLOOR = 0.35` sobre la similitud coseno del embedding — por debajo, la lección es de otro dominio y se ignora. "Nothing relevant enough → no advice beats wrong advice" (devuelve "", no ruido).
- **Recency decay:** `Math.exp(-ageDays / 45)` (half-life ~31 días) — lecciones nuevas pesan más, no hay consejo contradictorio de versiones viejas de la app.

13.4.2 — Skill drift → validation gate

El riesgo: el optimizador de skills (SkillOpt, `skill-evolution.ts`) propone un edit que mejora una tarea pero degrada el resto (overfitting), o deriva el skill doc hasta volverlo inútil.

El gate (`evolveSkill`, `skill-evolution.ts:86-112`):

- Edits **acotados**: máximo 3 add/delete/replace, net change ≤ 500 chars, doc final ≤ 1900 chars (el "learning rate" textual).
- **Held-out validation set** de 5 tareas FROZEN y diversas (rentals / swipe-marketplace / live scores / task manager / subscription box). Un edit se acepta **sólo si** `after > before + MARGIN` (`MARGIN` default 1.5) sobre el `objectiveScore` promedio del rollout en ese set.
- **Rejected-edit buffer:** `puglit_skill_rejects` — un edit rechazado se registra y NO se vuelve a proponer.
- Corre **offline** (SkillOpt-Sleep style), nunca inline por build → cero costo de inferencia extra en generación. El artefacto desplegado es el skill doc validado, vía `skillFor()`.

Tabla — Evolution Failures

Failure	Gate	Mecanismo	Dónde
Lesson poisoning	Quality filter	<code>quality >= 45 AND outcome <> 'failure'</code>	<code>progression.ts:161</code>
Cross-domain bleed	Relevance floor	<code>cosine >= 0.35</code> , si nada pasa → ""	<code>progression.ts:165–173</code>
Stale advice	Recency decay	<code>exp(-ageDays/45)</code>	<code>progression.ts:170</code>
Skill overfit / drift	Held-out validation gate	<code>after > before + 1.5</code> sobre 5 val tasks frozen	<code>skill-evolution.ts:101–109</code>
Edit thrashing	Rejected-edit buffer	<code>puglit_skill_rejects</code> , no re-proponer	<code>skill-evolution.ts:72–74, 110</code>

13.5 — Module Failures

Los ~85 módulos builtin se inyectan determinísticamente en finalize cuando matchean keywords. Dos modos de falla.

Tabla — Module Failures

Failure	Síntoma	Detección	Reparación
Phantom tables	El generado quereaa una tabla que nadie hizo <code>CREATE TABLE</code> (el bug "5 tablas inventadas" / FK-to-users)	<code>consistencyScan()</code> flag <code>phantom-table</code> (high) — <code>from/join/into/update <tabla></code> no declarada ni del spine ni <code>pg_*</code>	<code>repairPhantomTables()</code>
Dependency missing	El módulo necesita una extensión/sidecar (pgvector, ScrapeGraphAI) ausente	Falla al cargar SQL / al llamar el sidecar	Fallback degradado (JSONB en vez de <code>vector</code> ; sidecar opcional)

13.5.1 — Phantom-table repair, en detalle

`repairPhantomTables` (`swarm-repair.ts:41–69`) cierra el loop: `swarm-checks` DETECTA, esto REPARA. El cuidado de diseño está en no *persistir* la alucinación:

1. **Intención sobre uso:** prefiere el schema INTENCIONADO del arquitecto (ground truth) por sobre inferir columnas de una query posiblemente equivocada.
2. **Typo, no invención:** primero decide si la phantom table es un typo/alias de una tabla intencionada (entonces mapea / crea un `CREATE VIEW alias AS SELECT * FROM real_table`); sólo si es genuinamente nueva infiere columnas de cómo se usa.
3. **Reversible:** deja un backup comentado del SQL pre-reparación en `app.sql` ("auto-repair (swarm): phantom tables reconciled") para que una mala reparación sea auditable/reversible.

El spine siempre provee `users, sessions, accounts, analytics_events, password_resets, magic_links, records` (`SPINE_TABLES` , `swarm-checks.ts:14`) — referenciarlas NO es una alucinación.

13.6 — Infrastructure Failures

Puglit corre en un pod RunPod **A40 48GB**, con Postgres 14 + pgvector, Ollama (~77GB de modelos) y sidecars. La infra falla de las formas clásicas de un box con disco y RAM finitos.

Tabla — Infrastructure Failures

Failure	Síntoma	Causa	Mitigación cableada / Playbook
Disco lleno	<code>npm install</code> falla, SQL no carga, <code>cp</code> de <code>node_modules</code> cae	~77GB de modelos Ollama + builds + <code>node_modules</code> acumulados	<code>cp -al</code> (hard-links, 0 disco extra , inodes compartidos); build DIR junto al spine en el MISMO FS; limpiar <code>.builds/</code>
OOM (caja chica)	El modelo cuelga / mata el proceso en una caja de 8GB	KV cache del modelo no entra	<code>num_ctx</code> capeado (default 8192) "to keep the KV cache feasible locally"; streaming inmune a <code>headersTimeout</code>
Pod restart	Se pierde el estado en memoria	El pod se reinicia	La brain vive en cloud Postgres autoritativo + snapshots git/bucket → <code>brain-restore.sh</code> re-mergea en boot
Jobs colgados	Un job <code>running</code> que no avanza	Modelo lento / call colgado	El driver loopea hasta 400 iteraciones con backoff; un job ya <code>done</code> se saltea
Sidecar caído	ScrapeGraphAI no responde	Sidecar Python local caído	El módulo scraper degrada; el resto de la app no depende de él

13.6.1 — El detalle del disco y los hard-links

`build-local.mjs:166-175` es explícito: `node_modules` debe ser un **directorio REAL** (Turbopack rechaza un symlink que apunta afuera del proyecto), pero copiar 300MB por cada app servida es inviable con disco apretado. La solución: `cp -al` (hard-links — instantáneo, sin disco extra, inodes compartidos), con fallback a `cp -Rc` (clonefile) y por último `cp -R` real. Mismo filesystem obligatorio: por eso `DIR` vive bajo el repo, no en `/tmp`.

13.7 — Security Risks

El código generado por modelos chicos tiende a patrones inseguros. `securityScan` (`swarm-checks.ts:17-29`) hace un scan estático antes de entregar.

Tabla — Security Risks

Riesgo	Patrón detectado	Severidad	Reparación
Hardcoded secret	<code>sk-...</code> , <code>AKIA...</code> , <code>ghp_...</code> (key/token real comiteado)	high	Frontier escalation
Hardcoded credential	<code>password/secret/api_key/token = "<literal 8+ chars>"</code> sin <code>process.env</code>	med	Flag al critic
Dangerous exec	<code>eval(</code> , <code>new Function(</code> , <code>child_process</code> , <code>exec(</code> — riesgo RCE	high	Frontier escalation
SQL injection	<code>.query(\ ...\${...})</code> — SQL por interpolación de strings high Frontier escalation → parametriza <code>\$1,\$2</code>		
XSS	<code>dangerouslySetInnerHTML</code> sin <code>sanitize/DOMPurify/escapeHtml</code>	med	Flag al critic

13.7.1 — Frontier escalation (el techo del modelo local)

La crítica honesta: el auto-repair local sólo *flaggea* los issues de seguridad; un modelo chico no siempre los arregla bien. Por eso `repairSecurityWithFrontier` (`swarm-repair.ts:17-33`) gasta un **presupuesto acotado de un modelo MÁS FUERTE** para reescribir el archivo ofensor en los high de `sql-injection` / `hardcoded-secret` / `dangerous-exec` . Es **no-op salvo que `PUGLIT_FRONTIER_BUDGET` esté seteado** — por defecto el sistema es 100% local y sin costo; la escalada es opt-in.

13.7.2 — El Adversary lens

Más allá del scan estático, el `adversarial-review.ts` corre un lens **ADVERSARY** sesgado a RECHAZAR (`adversarial-review.ts:29`): busca SQLi, falta de auth / per-user scoping (IDOR), race conditions, double-booking/double-spend, overflow de precisión, input no validado llegando a la DB, secrets en código. Un hallazgo crítico confirmado por ≥ 2 lentes dispara una reparación acotada antes de entregar; 2 lentes en `reject` → veredicto **BLOCK**.

Riesgo residual honesto: estos son scans heurísticos (regex + LLM lenses), no un análisis formal. Detectan las clases comunes, no garantizan ausencia de vulnerabilidades. La app generada es un punto de partida auditable, no un sistema certificado.

13.8 — Dependency Risks

Riesgo	Impacto	Mitigación
El spine fija las deps	Toda app generada hereda <code>next / react / pg</code> del spine; un CVE en el spine se propaga	Superficie chica y auditable: el spine es deps base, no un zoo de paquetes
Imports inventados	El modelo importa de <code>@/lib/<x></code> inexistente	<code>consistencyScan</code> flag <code>missing-import</code> ; el repair NO agrega npm deps ("Add NO npm deps", <code>build-local.mjs:281</code>)
Deps prohibidas en repair	El modelo intenta <code>npm install</code> algo nuevo durante una reparación	Todos los system prompts de repair: "Import only from next/, react, @/lib/, @/domain.config"
Modelos no pulleados	El council no arranca (team model ausente)	Fallback a <code>MODELS.code</code> ; si cero designs → error claro "¿modelos del council bajados?"
pgvector / sidecars opcionales	RAG/scrapper requieren extensión/proceso externo	Degradación: JSONB en vez de <code>vector</code> , sidecar opcional

El principio: **el generado nunca introduce dependencias nuevas**. La reparación trabaja con la superficie fija del spine (`next/*`, `react`, `@/lib/*`, `@/domain.config`). Esto acota drásticamente el blast radius de un problema de dependencias — no hay `node_modules` arbitrario que cada app pueda traer.

13.9 — Scaling Risks

Puglit corre sobre **1 GPU**. Eso impone la limitación de escala más importante y varias derivadas.

Tabla — Scaling Risks

Riesgo	Causa	Realidad / Mitigación
Council secuencial	1 GPU = la cola está compartida; los 3 teams + 3 modelos corren uno por uno	"On 1 GPU the 3 teams + models run SEQUENTIALLY (shared queue)" — el throughput es ~3× el de un single-model; no es paralelo real
Jurado secuencial	Multi-juror panel también swapea modelos en la misma GPU	"sequential: 1 GPU swaps" — más jurors = más latencia, no más paralelismo
Token cost del judging	Juzgar 75 agentes uno por uno sería carísimo en tokens	El Gran Jurado puntúa por TEAM y por ÁREA (no agente por agente); la distribución de XP es local/aritmética
Context window chico	Modelos locales tienen ventanas chicas	El <code>digest()</code> del adversarial review capea a ~24KB de los archivos domain-críticos; <code>num_ctx</code> capeado
Builds concurrentes en parallel pods	Dos pods evolucionando la brain a la vez podrían contradecirse	<code>brain-sync.ts</code> : append-only por UNION (CRDT grow-set), mutable arbitrado por <code>val_score</code> — nunca last-write-wins
Brain crece sin techo	Diary/metrics/exemplars son append-only	<code>relevantLessons</code> limita a 400 filas recuperadas + <code>DISTINCT ON (entry)</code> ; el recall es por relevancia, no full-scan

13.9.1 — El cuello de botella de 1 GPU

Es honesto decirlo: la limitación dominante de escala es la GPU única. El multi-modelo council y el multi-juror panel son **ensembles secuenciales**, no paralelos. Más diversidad (más teams, más jurors) compra mejor señal a costa de latencia lineal. La arquitectura está lista para paralelizar sobre múltiples GPUs/pods — la brain ya es merge-aware y CRDT-friendly (§13.10) — pero el deployment de referencia es 1 pod, 1 A40.

13.10 — Recovery Playbooks

Las secuencias exactas para volver a verde. Cada playbook es ejecutable contra el código de hoy.

Playbook A — Resumable training (el build local no converge)

Síntoma: SQL no carga / `tsc` no compila / smoke test falla.

1. El pipeline **ya** reintenta solo: SQL hasta 5 rondas (`loadAppSchemaRepairing`), `tsc` hasta `MAX_ROUNDS` (default 6) con anti-oscilación y restauración del best-seen.
2. **Reusar el job, no regenerar:** `JOB_ID=<id> node web/scripts/build-local.mjs` — un job ya `done` se saltea el driving (`build-local.mjs:101-102`); el `createAndDrive` es idempotente.
3. Si SQL no converge, el build **sigue igual** con tablas parciales — la app corre con lo que cargó. El smoke test te dice qué páginas dan 5xx (`failing: [...]` en el status).
4. Subir el bar o bajar presión: `MAX_ROUNDS` , `PUGLIT_COVERAGE_BAR` (default 70), `PUGLIT_QA_ROUNDS` (default 2) son env-tunables.

Playbook B — Brain-restore merge (pod fresco / offline / corrupto)

Síntoma: pod nuevo sin brain, o sospecha de brain inconsistente.

1. `psql genetic.sql` (crea las tablas de la brain).
2. `SVC=$PUGLIT_SERVICE_TOKEN BRAIN_REPO=/path/to/puglit-brain bash infra/brain-restore.sh` — fetchea el último snapshot y lo **MERGEA** (no clobbera) vía `POST /api/admin/brain` → `mergeBrain()` .
3. El merge respeta dos clases (`brain-sync.ts:54-81`): - **Append-only** (diary, metrics, exemplars, skill-rejects) → UNION por content key (nada se sobrescribe). - **Mutable** (skills, modules, agent XP) → arbitrado por score objetivo: skill activo = mejor `val_score` held-out; módulo = mayor versión; XP = `GREATEST` (nunca se pierden niveles).
4. `consolidateActiveSkills()` corre al final: por área, el skill activo = el de mayor `val_score` .
5. Es **seguro correrlo en cualquier momento** — nunca pisa una brain más nueva. Idealmente en cada pod start, después de `genetic.sql` .

Backup proactivo: `bash infra/brain-snapshot.sh` (en cron / después de cada run) exporta a `BRAIN_REPO/latest.json` + `snapshots/brain-<TS>.json` y pushea a git. El store vivo autoritativo es siempre la cloud Postgres; el snapshot es belt-and-suspenders.

Playbook C — Reset --hard (una reparación envenenó el código)

Síntoma: un auto-repair (phantom-table, security, adversarial) dejó el código peor.

1. **Phantom-table:** el backup comentado está EN `app.sql` ("pre-repair backup ... original SQL preserved above this marker", `swarm-repair.ts:63`). La reparación es reversible por inspección.

- 2. **tsc repair:** el `repairTs` ya restaura el **best-seen snapshot** si el estado final es peor que el mejor visto (`build-local.mjs:286-288`) — no necesitas intervenir, lo hace solo.
- 3. **Adversarial repair:** es **bounded** — máximo 3 archivos críticos, "MINIMAL change", no reescribe partes que funcionan (`adversarial-review.ts:86-101`). En `PUGLIT_TRAINING_MODE` se saltea el repair + re-review entero (review once, batch brain-fill).
- 4. **Skill envenenado:** no hay reset porque no hay write sin gate — un edit malo nunca pasó el held-out gate; quedó en `puglit_skill_rejects`. El skill activo siempre es el de mejor `val_score`.
- 5. **Último recurso (job entero):** el código generado vive en `puglit_jobs.artifacts` (Postgres); borrar `.builds/puglit-<slug>` y re-correr con un `JOB_ID` distinto regenera limpio sin tocar la brain.

Tabla — Recovery Playbooks (resumen)

Escenario	Playbook	Comando / mecanismo clave	Garantía
Build no converge	A — Resumable training	<code>JOB_ID=<id> node build-local.mjs</code>	Idempotente; reintentos acotados; best-seen restore
Pod fresco / brain dudosa	B — Brain-restore merge	<code>brain-restore.sh</code> → <code>mergeBrain()</code>	UNION + arbitraje objetivo; nunca clobbera
Repair envenenó el código	C — Reset --hard	backup en <code>app.sql</code> / best-seen / rejected buffer	Reversible o auto-restaurado; skill nunca degrada

13.11 — Resumen: la filosofía de falla

Tres invariantes recorren todo este capítulo:

- 1. **Degradar, no abortar.** Casi ningún fallo individual tira el pipeline. Un agente que se niega se saltea; un modelo que falla cae a fallback; un jurado caído activa el circuit breaker; un SQL que no converge corre con tablas parciales. El sistema prioriza *entregar algo verificable* por sobre *fallar limpio*.
- 2. **El runtime gate es la verdad.** Static scans (security, consistency, adversarial lenses) son necesarios pero no suficientes. La única prueba de que la app funciona es **bootearla y pegarle GET sin 5xx**. Los 7 bugs de Stayforge lo demuestran: invisibles a un compile verde, obvios al primer request.
- 3. **El aprendizaje está gateado.** Nada entra a la brain sin pasar un filtro objetivo: lecciones por quality/relevance/recency, skills por held-out validation, merges por score y no por last-write. El sistema puede aprender lento — pero le cuesta aprender mal.

El riesgo residual honesto: Puglit corre modelos chicos sobre 1 GPU; las defensas son heurísticas (regex + LLM lenses + gates determinísticos), no verificación formal. El generado es un punto de partida auditable y *que arranca*, no un sistema certificado para producción sin revisión humana.